ELI — Diagrams

ELI v2.1.66

August 2026

Contents

Blueprint — ELI MKXI ASCII Diagrams

Visual companion to `architecture.md`. Three views: the full request pipeline, the memory subsystem, and the gating stack. All grounded in the real modules (file paths are clickable).

---

1. Full pipeline (request lifecycle)

```
        voice ──► STT (faster-whisper)          GUI text
          │        perception/local_whisper_stt     │
          └───────────────┬─────────────────────────┘
                          ▼  raw text
            ┌─────────────────────────────────┐
            │  ROUTER                         │  execution/router_enhanced.py
            │  priority pipeline (regex-first)│  → {action, args, conf, matched_by}
            │  LLM-intent fallback            │     cognition/llm_intent.py
            └─────────────┬───────────────────┘
                          ▼
            ┌─────────────────────────────────┐
            │  CognitiveEngine.process()      │  kernel/engine.py
            └─────────────┬───────────────────┘
        ┌─────────────────┼─────────────────────────────────┐
        ▼                 ▼                                  ▼
╔══════════════╗ ╔══════════════════════╗        ╔══════════════════════╗
║ [A] PHASE45  ║ ║ [B] ORCHESTRATOR     ║        ║ [C] QUICK PATH       ║
║ FAST-PATH    ║ ║ (deep, non-quick)    ║        ║ (default, CHAT)      ║
║ OS/media/    ║ ║ full 12-stage        ║        ║                      ║
║ status/job   ║ ║ (see §1b)            ║        ║   AgentBus.dispatch()║
╚══════════════╝ ╚══════════════════════╝        ╚══════════════════════╝
        │ execute_action()     │ assembled context           │ DispatchResult
        │ VERBATIM (no LLM)    │                             │  • grounding_confidence
        │ can't confabulate    │                             │  • memory_context
        ▼                      │                             ▼
```

```
   return                  │
     ▲                     │                     ┌────────────────────────────┐
   │                │             │  factual? + low grounding?  │  GROUNDING ESCALATION       │  (CHAT only)
     │                     │                     │  runtime/grounding_escalation.py
     │                     │                     └────────────────────────────┘
     │                     │                         yes  │            │ no
     │                     │            external→WEB / local→agents │ proceed
     │                     │            else → HEDGE ("won't guess")│
     │                     │                        │  return       ▼
     │                     │                        │      ┌────────────────────┐
    │                │                  │     │ self-contained?   │─► return executor verbatim
    │                  │                  │     │ grounded control? │─► grounded synthesis
     │                     │                        │      │ plain CHAT          │
     │                     │                        │      └────────────────────┘
     │                     ▼                        │             ▼
     │          _build_enhanced_system()  ◄─────────┘      persona(budget-trimmed)
     │          (persona + evidence + brief, n_ctx budget)   + situation brief
     │                     │                                      │
     │                     ▼                                      ▼
     │           inference_broker.infer() / stream  ── gguf_inference (model-agnostic)
     │                     │                                      │
     │                     ▼                                      ▼
     │          CONFIDENCE GATE (Stage 12)  ── score vs threshold → PASS / repair
     │                     │
     │                     ▼
     └────────────►  OUTPUT GOVERNOR / sanitiser ── cognition/output_governor.py
                           │   (strip leaks, persona hygiene, no raw packets)
                           ▼
                 response ──► TTS (Piper)  /  GUI
```

1b. Orchestrator — the 12 stages (path [B]) `cognition/orchestrator.py`

```
 1 Intent ─► 2 Persona Lock ─► 3 HyDE ─► 4 Planner
                                            │
     ┌──────────────────────────────────────┘
     ▼     5/6/7  PARALLEL RETRIEVAL
  ┌─────────┬─────────┬─────────┬─────────┬─────────┐
  │ keyword │  FTS5   │  FAISS  │   RAG   │   KG    │
  └─────────┴─────────┴─────────┴─────────┴─────────┘
     │         │         │         │         │
     └─────────┴─────────┴─────────┴─────────┘
                    ▼
     8 Hybrid Merge ─► 9 Rerank ─► 10 Context Assembly
                   ─► 10.5 Persona Handoff ─► 11 LLM Generation ─► 12 Confidence
```

---

2. Memory subsystem `eli/memory` + `cognition/working_memory.py`

```
                          query / turn
                               │
     ┌─────────────────────────┼────────────────────────────┐
     ▼          ▼              ▼            ▼             ▼
  ┌─────────┐ ┌─────────┐ ┌──────────┐ ┌─────────┐ ┌──────────────┐
  │ keyword │ │  FTS5   │ │  FAISS   │ │  RAG    │ │ KnowledgeGraph│
  │ (SQL)   │ │ full-txt│ │ vectors  │ │ docs    │ │ entities/rel  │
  └─────────┘ └─────────┘ └──────────┘ └─────────┘ └──────────────┘
     │          │              │            │            │
```

```
      ▼          ▼          ▼           ▼           ▼
conversation  memories_fts  index.faiss          kg_entities(+_fts)
_turns        memories      (vectors/)            kg_relations
(user.sqlite3)              embedder:
                            nomic-embed-…Q4_K_M.gguf
      └──────────┴────┬─────┴───────────┴───────────┘
                      ▼
           HYBRID MERGE ─► RERANK ─► assembled context
                      │
           + working-memory pins (session-pinned, junk-guarded)
                      ▼
              situation brief ─► generation (persona handoff)

┌─────────────────────── STORES (artifacts/) ───────────────────────┐
│ db/user.sqlite3   memories(+_fts), conversation_turns, conversations,  │
│                   kg_entities(+_fts), kg_relations, recall_log,        │
│                   runtime_events, news_articles(+_fts), news_reflections,│
│                   habits/habit_events/habit_rules, observations,       │
│                   learning_replay, working_memory_pins, user_patterns, │
│                   session_summaries, corrections, failures             │
│ db/agent.sqlite3  agent_dispatches, agent_metrics, improvements,       │
│                   failures, code_patches, error_tracking, observations │
│ vectors/index.faiss   semantic index                                   │
│ runtime/users/<uuid>/user_profile…   per-user profile                  │
└───────────────────────────────────────────────────────────────────────┘

WRITE PATH:  turn ─► PERSISTENCE GATE (drop junk/report-dumps) ─► store
```

---

3. Gating stack (in path order — every guard the request passes)

```
┌────────────────────────────────────────────────────────────────────┐
│ 1 ░ NETGUARD  (process-wide, always-on)       core/netguard.py          │
│      any outbound socket ──► offline? ──► OfflineError (FAIL-CLOSED)     │
│      allow_network(): scoped opt-in window (model download, web tier)    │
├────────────────────────────────────────────────────────────────────┤
│ 2 ░ PHASE45 FAST-PATH GATE                    kernel/engine.py           │
│      deterministic action (volume/media/date/job/status)?                │
│        ──► execute_action() VERBATIM, no LLM  (cannot confabulate)       │
├────────────────────────────────────────────────────────────────────┤
│ 3 ░ PERSISTENCE GATE                          runtime/persistence_gate.py│
│      about to write to memory? junk / internal report-dump?              │
│        ──► DROP (so it can't be stored and replayed as fact later)       │
├────────────────────────────────────────────────────────────────────┤
│ 4 ░ GROUNDING GATE                            runtime/deterministic_…gate│
│      answer requires evidence? unbacked or raw packet?                   │
│        ──► BLOCK raw evidence/telemetry leak; require grounded synthesis │
├────────────────────────────────────────────────────────────────────┤
│ 5 ░ GROUNDING ESCALATION                      runtime/grounding_escalation│
│      checkable fact  AND  grounding < threshold ?                        │
│        external fact ─► WEB tier ────┐                                   │
│        self/project  ─► LOCAL tier ──┼─► first tier that grounds ─► answer│
│        exhausted / offline ──────────┘─► HEDGE  ("I won't guess")        │
│      (trigger = GROUNDING, not the response score — which lies when wrong)│
├────────────────────────────────────────────────────────────────────┤
```

```
│ 6 ░ CONFIDENCE GATE  (Stage 12)                kernel/engine.py              │
│      response score vs threshold ──► PASS  /  repair pass                      │
├──────────────────────────────────────────────────────────────────────────────┤
│ 7 ░ OUTPUT GOVERNOR / SANITISER                cognition/output_governor.py │
│      final text ──► strip leaks, persona hygiene, no raw JSON ──► user         │
└──────────────────────────────────────────────────────────────────────────────┘

Legend:  FAIL-CLOSED = denies by default;  VERBATIM = returned without an LLM
         pass;  HEDGE = honest "can't verify" instead of a guess.
```

---

4. One-screen overview (how it all connects)

```
INPUT ─► ROUTER ─► ENGINE ─┬─ FAST-PATH ──────────────► (verbatim) ─► OUTPUT
                           ├─ ORCHESTRATOR ─► retrieval ┐
                           └─ QUICK ─► AGENT BUS ───────┤
                                                        ▼
                        MEMORY (SQL/FTS5/FAISS/KG/WM) ─► context
                                                        ▼
                        GATES: netguard · persistence · grounding ·
                               escalation · confidence · governor
                                                        ▼
                        INFERENCE (broker → gguf, model-agnostic) ─► OUTPUT
                                                        ▼
                        background: proactive · self-improve · habits ·
                                    learning(LoRA) · world · scheduler
```

“‘